In [1]:

```
# Dogs vs Cats - DCNN Classification

import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import os
%matplotlib inline
```

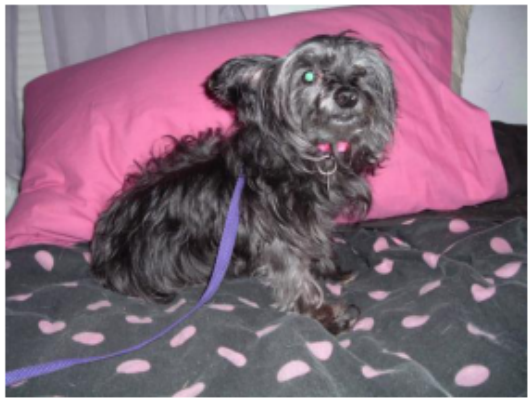
In [2]:

```
# The images are in the data/shapes folder
data_folder = 'C:/Users/ABHISHEK JAYAN/Downloads/PetImages'

# Get the class names
classes = os.listdir(data_folder)
classes.sort()
print(len(classes), 'classes:')
print(classes)

# Show the first image in each folder
fig = plt.figure(figsize=(8, 12))
i = 0
for sub_dir in os.listdir(data_folder):
    i+=1
    img_file = os.listdir(os.path.join(data_folder,sub_dir))[0]
    img_path = os.path.join(data_folder, sub_dir, img_file)
    img = mpimg.imread(img_path)
    a=fig.add_subplot(1, len(classes),i)
    a.axis('off')
    imgplot = plt.imshow(img)
    a.set_title(img_file)
plt.show()
```

```
2 classes:
['Cat', 'Dog']
```

0.jpg

0.jpg



In [3]:
```
from PIL import Image
import os

def remove_corrupted_images(directory):
    for root, dirs, files in os.walk(directory):
        for file in files:
            file_path = os.path.join(root, file)
            try:
                img = Image.open(file_path)
                img.verify()
            except Exception:
                print("Removing corrupted file:", file_path)
                os.remove(file_path)

# Run once BEFORE generators
remove_corrupted_images(data_folder)
```

```
D:\Anaconda\lib\site-packages\PIL\TiffImagePlugin.py:845: UserWarning: Trunc
ated File Read
  warnings.warn(str(msg))
```

In [4]:
```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

img_size = (128, 128)
batch_size = 30

print("Getting Data...")
datagen = ImageDataGenerator(rescale=1./255, # normalize pixel values
                             validation_split=0.3) # hold back 30% of the imag

print("Preparing training dataset...")
train_generator = datagen.flow_from_directory(
    data_folder,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical',
    subset='training') # set as training data

print("Preparing validation dataset...")
validation_generator = datagen.flow_from_directory(
    data_folder,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation') # set as validation data

classnames = list(train_generator.class_indices.keys())
print('Data generators ready')
```

```
Getting Data...
Preparing training dataset...
Found 17500 images belonging to 2 classes.
Preparing validation dataset...
Found 7498 images belonging to 2 classes.
Data generators ready
```

In [12]:
```
train_generator.class_indices
```

Out[12]:
```
{'Cat': 0, 'Dog': 1}
```

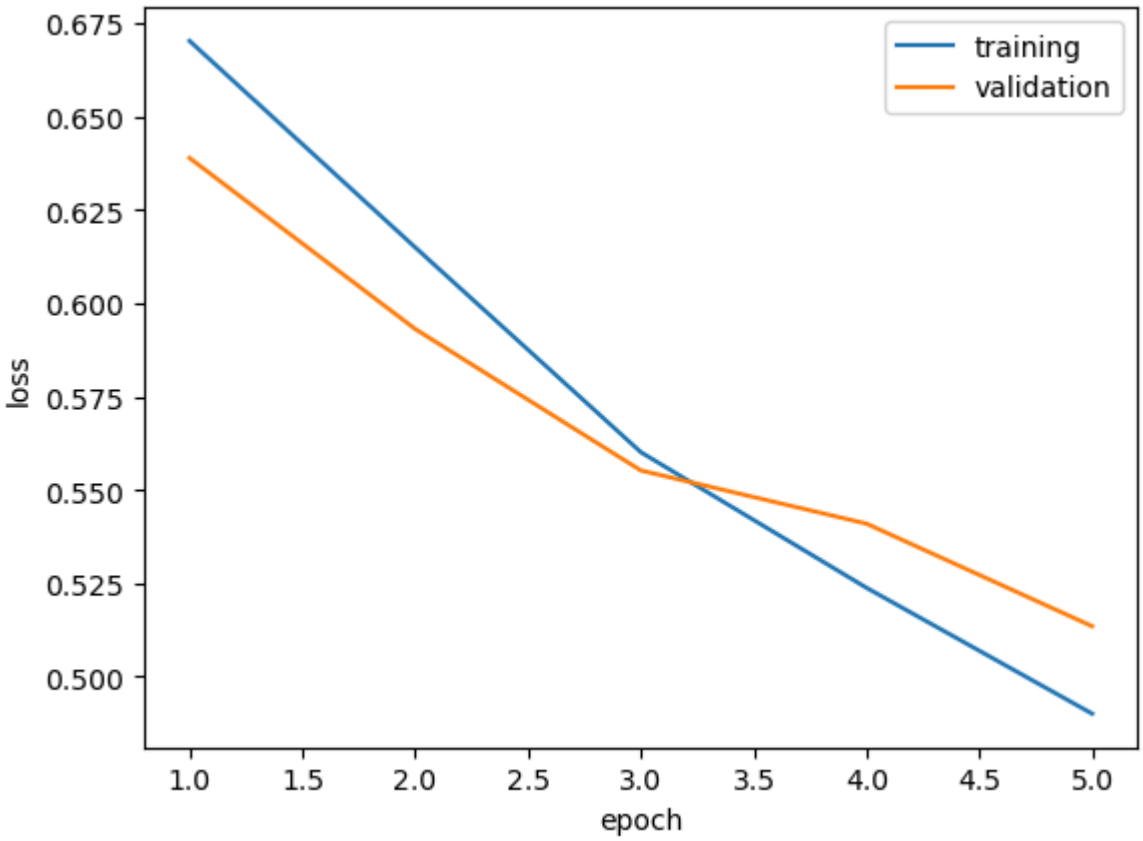
In [5]:
```
# Define a CNN classifier network
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dropout, Flatten, De

# Define the model as a sequence of layers
model = Sequential()

# The input layer accepts an image and applies a convolution that uses 32 6x6
model.add(Conv2D(32, (6, 6), input_shape=train_generator.image_shape, activati

# Next we'll add a max pooling layer with a 2x2 patch
model.add(MaxPooling2D(pool_size=(2,2)))

# We can add as many layers as we think necessary - here we'll add another con
model.add(Conv2D(32, (6, 6), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# And another set
model.add(Conv2D(32, (6, 6), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# A dropout layer randomly drops some nodes to reduce inter-dependencies (whic
model.add(Dropout(0.2))

# Flatten the feature maps
model.add(Flatten())

# Generate a fully-connected output layer with a predicted probability for eac
# (softmax ensures all probabilities sum to 1)
model.add(Dense(train_generator.num_classes, activation='softmax'))

# With the layers defined, we can now compile the model for categorical (multi
model.compile(loss='categorical_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])

print(model.summary())
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 123, 123, 32)      3488

 max_pooling2d (MaxPooling2D  (None, 61, 61, 32)       0
 )

 conv2d_1 (Conv2D)           (None, 56, 56, 32)        36896

 max_pooling2d_1 (MaxPooling  (None, 28, 28, 32)       0
 2D)

 conv2d_2 (Conv2D)           (None, 23, 23, 32)        36896

 max_pooling2d_2 (MaxPooling  (None, 11, 11, 32)       0
 2D)

 dropout (Dropout)           (None, 11, 11, 32)        0

 flatten (Flatten)           (None, 3872)              0

 dense (Dense)               (None, 2)                 7746

=================================================================
Total params: 85,026
Trainable params: 85,026
Non-trainable params: 0
_________________________________________________________________
None
```

In [6]:
```
# Train the model over 5 epochs using 30-image batches and using the validatic
num_epochs = 5
history = model.fit(
    train_generator,
    steps_per_epoch = train_generator.samples // batch_size,
    validation_data = validation_generator,
    validation_steps = validation_generator.samples // batch_size,
    epochs = num_epochs)
```

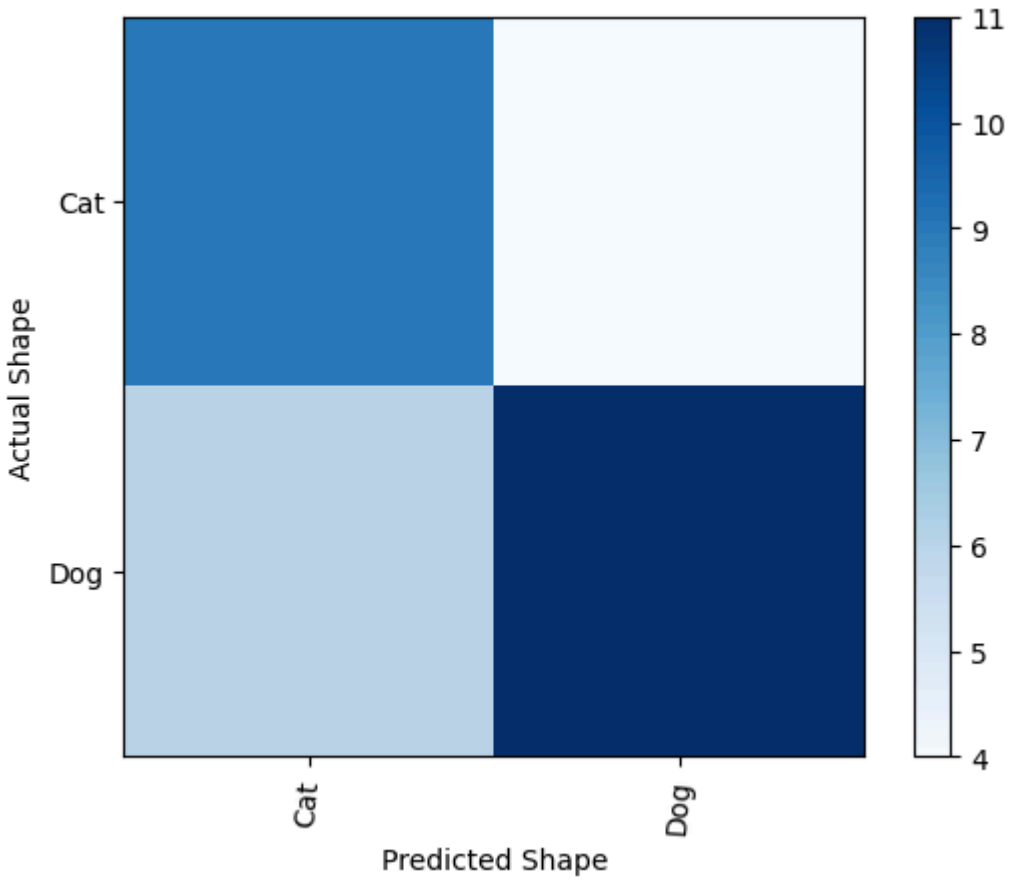
```
Epoch 1/5
254/583 [============>.................] - ETA: 3:38 - loss: 0.6853 - accura
cy: 0.5466

D:\Anaconda\lib\site-packages\PIL\TiffImagePlugin.py:845: UserWarning: Trunc
ated File Read
  warnings.warn(str(msg))

583/583 [==============================] - 447s 765ms/step - loss: 0.6702 -
accuracy: 0.5851 - val_loss: 0.6389 - val_accuracy: 0.6414
Epoch 2/5
583/583 [==============================] - 636s 1s/step - loss: 0.6150 - acc
uracy: 0.6626 - val_loss: 0.5931 - val_accuracy: 0.6871
Epoch 3/5
583/583 [==============================] - 475s 815ms/step - loss: 0.5601 -
accuracy: 0.7160 - val_loss: 0.5552 - val_accuracy: 0.7191
Epoch 4/5
583/583 [==============================] - 428s 735ms/step - loss: 0.5238 -
accuracy: 0.7454 - val_loss: 0.5410 - val_accuracy: 0.7324
Epoch 5/5
583/583 [==============================] - 426s 730ms/step - loss: 0.4901 -
accuracy: 0.7678 - val_loss: 0.5135 - val_accuracy: 0.7483
```

In [7]:
```
%matplotlib inline
from matplotlib import pyplot as plt

epoch_nums = range(1,num_epochs+1)
training_loss = history.history["loss"]
validation_loss = history.history["val_loss"]
plt.plot(epoch_nums, training_loss)
plt.plot(epoch_nums, validation_loss)
plt.xlabel('epoch')
plt.ylabel('loss')
plt.legend(['training', 'validation'], loc='upper right')
plt.show()
```



In [8]:
```
# Tensorflow doesn't have a built-in confusion matrix metric, so we'll use Sci
import numpy as np
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
%matplotlib inline

print("Generating predictions from validation data...")
# Get the image and label arrays for the first batch of validation data
x_test = validation_generator[0][0]
y_test = validation_generator[0][1]

# Use the model to predict the class
class_probabilities = model.predict(x_test)

# The model returns a probability value for each class
# The one with the highest probability is the predicted class
predictions = np.argmax(class_probabilities, axis=1)

# The actual labels are hot encoded (e.g. [0 1 0], so get the one with the val
true_labels = np.argmax(y_test, axis=1)

# Plot the confusion matrix
cm = confusion_matrix(true_labels, predictions)
plt.imshow(cm, interpolation="nearest", cmap=plt.cm.Blues)
plt.colorbar()
tick_marks = np.arange(len(classnames))
plt.xticks(tick_marks, classnames, rotation=85)
plt.yticks(tick_marks, classnames)
plt.xlabel("Predicted Shape")
plt.ylabel("Actual Shape")
plt.show()
```

```
Generating predictions from validation data...
1/1 [==============================] - 0s 438ms/step
```



In [9]:
```
# Save the trained model
modelFileName = 'models/animal_classifier.h5'
model.save(modelFileName)
del model  # deletes the existing model variable
print('model saved as', modelFileName)
```

```
model saved as models/animal_classifier.h5
```

In [19]:
```
from tensorflow.keras.preprocessing import image
import numpy as np
import matplotlib.pyplot as plt

# Load image properly
img_path = "cat.jpg"

img = image.load_img(img_path, target_size=(128, 128))
img_array = image.img_to_array(img)

plt.axis('off')
plt.imshow(img)
plt.show()

# Expand dimensions (model expects batch)
img_array = np.expand_dims(img_array, axis=0)

# Normalize (because training used rescale=1./255)
img_array = img_array.astype('float32') / 255

# Load model
model = models.load_model(modelFileName)


# Predict
prediction = model.predict(img_array)

print("Raw prediction:", prediction)

# If binary classification (Dogs vs Cats)
print(train_generator.class_indices)

if prediction[0][1] > 0.5:
    print("Prediction: Dog")
else:
    print("Prediction: Cat")
```

```
1/1 [==============================] - 0s 94ms/step
Raw prediction: [[0.75635487 0.2436451 ]]
{'Cat': 0, 'Dog': 1}
Prediction: Cat
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: